Guia para el Examen Parcial 2

Desarrollo e Integracion de Aplicaciones con Bases de Datos NoSQL en la Nube — Sistema de reservacion de boletos con bloqueo temporal de asientos (Redis Cloud, Clave-Valor)

---

1. Uno o dos dias antes del examen

Enciende la laptop, conecta a internet y corre una vez el flujo completo para confirmar que tu base de Redis Cloud sigue activa (las bases gratuitas se pueden suspender despues de muchos dias sin uso).

```
npm run seed npm start
```

2. El dia del examen: preparacion antes de tu turno

1. Enciende la laptop y conectate a internet (la app usa Redis Cloud, no es local).
2. Abre VS Code: File > Open Folder > C:\Users\marco\Anahuac2025\Ingenieria\NoSQL-Parcial2
3. Abre una terminal y corre **npm run seed** para dejar los 25 asientos en estado 'available' (borra cualquier reserva de tus pruebas anteriores).
4. En esa misma terminal corre **npm start** y confirma que aparezca 'Connected to Redis Cloud' y 'Server running on port 3000'. No cierres esta terminal durante tu exposicion.
5. Abre una SEGUNDA terminal para los comandos de la demo (ver seccion 3).
6. Ten ya abiertas las pestanas del navegador con tu repositorio de GitHub y el PDF de diseno.

3. Demostracion en vivo (4 minutos)

Paso 1 — Mostrar los asientos disponibles:

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats"
```

> Di: "Aqui se ven los 25 asientos del evento, todos disponibles."

Paso 2 — Bloquear un asiento (TTL):

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A1-1/hold" -Method POST
-ContentType "application/json" -Body '{"customerId":"cust-123"}'
```

> Di: "Esto bloquea el asiento A1-1 por 300 segundos mientras el cliente paga. Si no confirma a tiempo, Redis lo libera solo."

Paso 3 — Confirmar la reservacion:

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A1-1/confirm" -Method POST
-ContentType "application/json" -Body '{"customerId":"cust-123"}'
```

> Di: "Esto confirma la reservacion de forma permanente." Vuelve a correr el comando del Paso 1 para mostrar que A1-1 ahora aparece como 'booked': eso demuestra la persistencia real en la nube.

4. Defensa tecnica (4-6 minutos)

Por que Clave-Valor (Redis) y no SQL relacional:

> "El bloqueo temporal necesita expirar solo despues de X segundos. En SQL eso requeriria un proceso externo revisando constantemente cuales bloqueos ya vencieron, con riesgo de que dos personas reserven el mismo asiento al mismo tiempo. En Redis, el comando SET con las opciones NX y EX crea el bloqueo y su expiracion en una sola operacion atomica: solo una persona puede quedarse con el asiento, y Redis lo libera solo, sin que yo programe nada extra."

Si el servidor se cae mientras un asiento esta bloqueado:

> "No importa: el TTL vive dentro de Redis, no en mi servidor, asi que el asiento se libera solo aunque mi aplicacion este caida."

Si dos personas intentan el mismo asiento al mismo tiempo:

> "La opcion NX hace que solo la primera peticion tenga exito; la segunda recibe un error de conflicto (409), evitando la doble reserva."

Convencion de llaves (namespacing) en Redis:

Llave	Tipo	Contenido
event:{eventId}	Hash	name, venue, date, price
event:{eventId}:seats	Set	ids de todos los asientos
seat:{eventId}:{seatId}	Hash	status, section, row
hold:{eventId}:{seatId}	String + TTL	customerId del bloqueo temporal
booking:{bookingId}	Hash	eventId, seatId, customerId, status
waitlist:{eventId}	Sorted Set	customerId, score = timestamp

5. Cierre

Menciona que los datos (evento, asientos, reservaciones confirmadas) no tienen TTL y quedan guardados permanentemente en Redis Cloud. Cierra mostrando el codigo en tu repositorio de GitHub y el documento de diseno (PDF) con la justificacion completa.